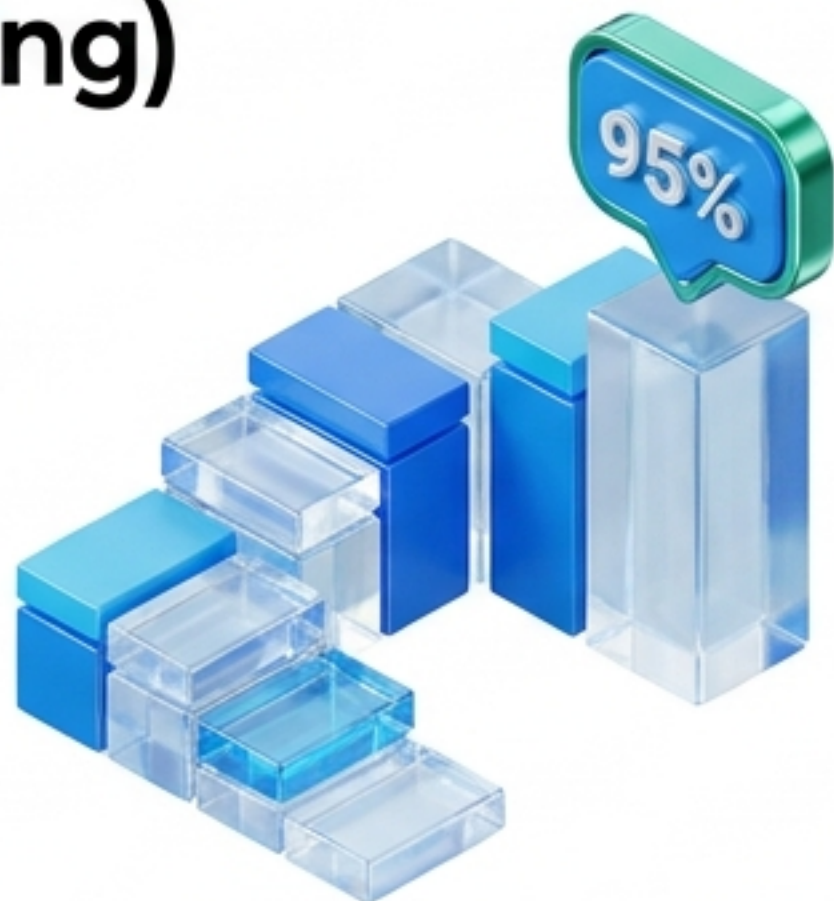


Tìm số chiều để giữ 95% phương sai (Phương pháp thủ công)



Chạy PCA mà không giảm chiều để lấy toàn bộ các PCs. Sau đó, tính toán lũy kế phương sai.

```
from sklearn.datasets import fetch_openml
import numpy as np

mnist = fetch_openml('mnist_784', as_frame=False)
X_train = mnist.data[:60_000]

pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
# d bằng 154
```

d = 154

Cách Scikit-Learn giải quyết: Truyền thẳng tỷ lệ phần trăm



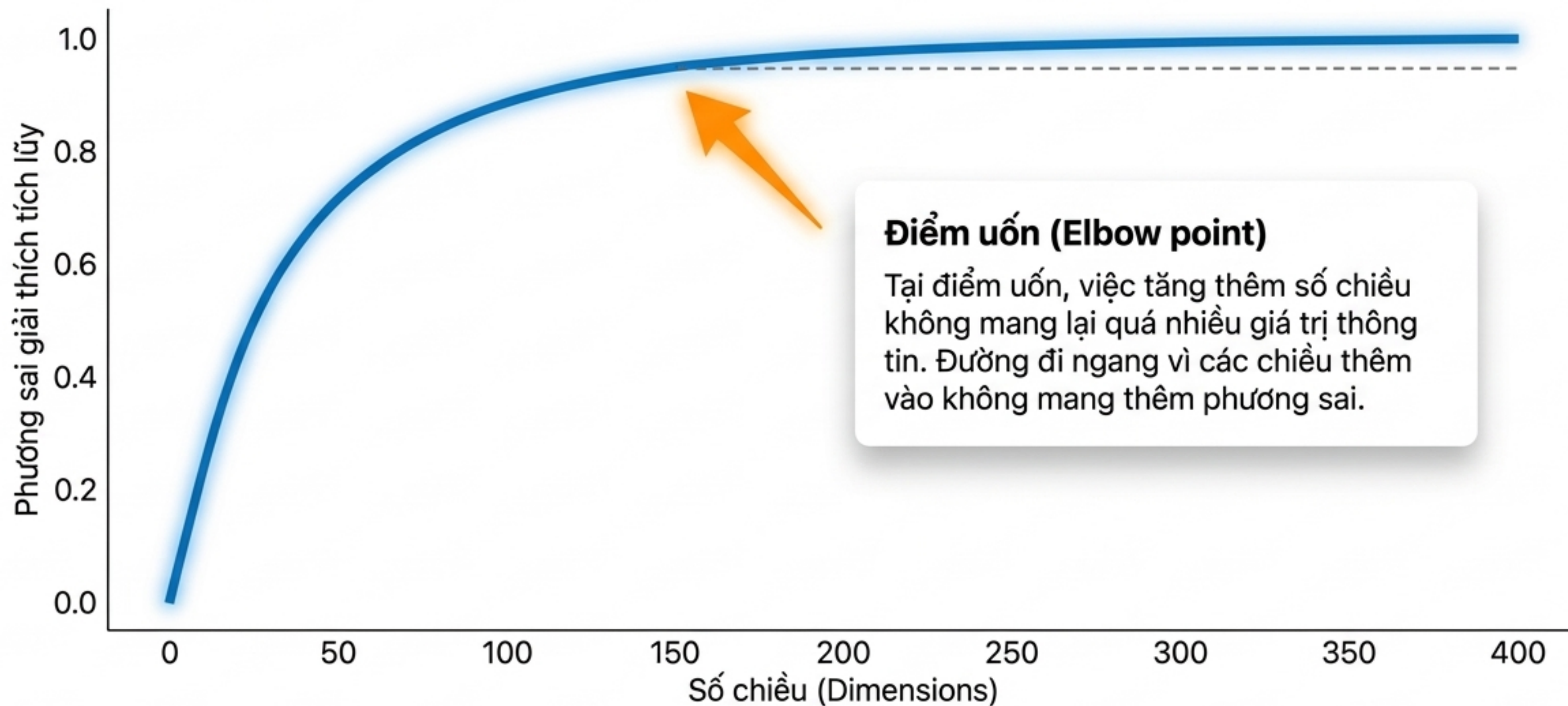
Một cách tốt hơn (và nhàn hơn) là truyền thẳng mức độ phương sai bạn muốn bảo toàn dưới dạng một số thập phân từ 0.0 đến 1.0 cho **n_components**. Scikit-learn sẽ tự tính toán số chiều tối thiểu.

```
# Scikit-learn sẽ tự tính toán số chiều tối thiểu
pca = PCA(n_components=0.95)

X_reduced = pca.fit_transform(X_train)

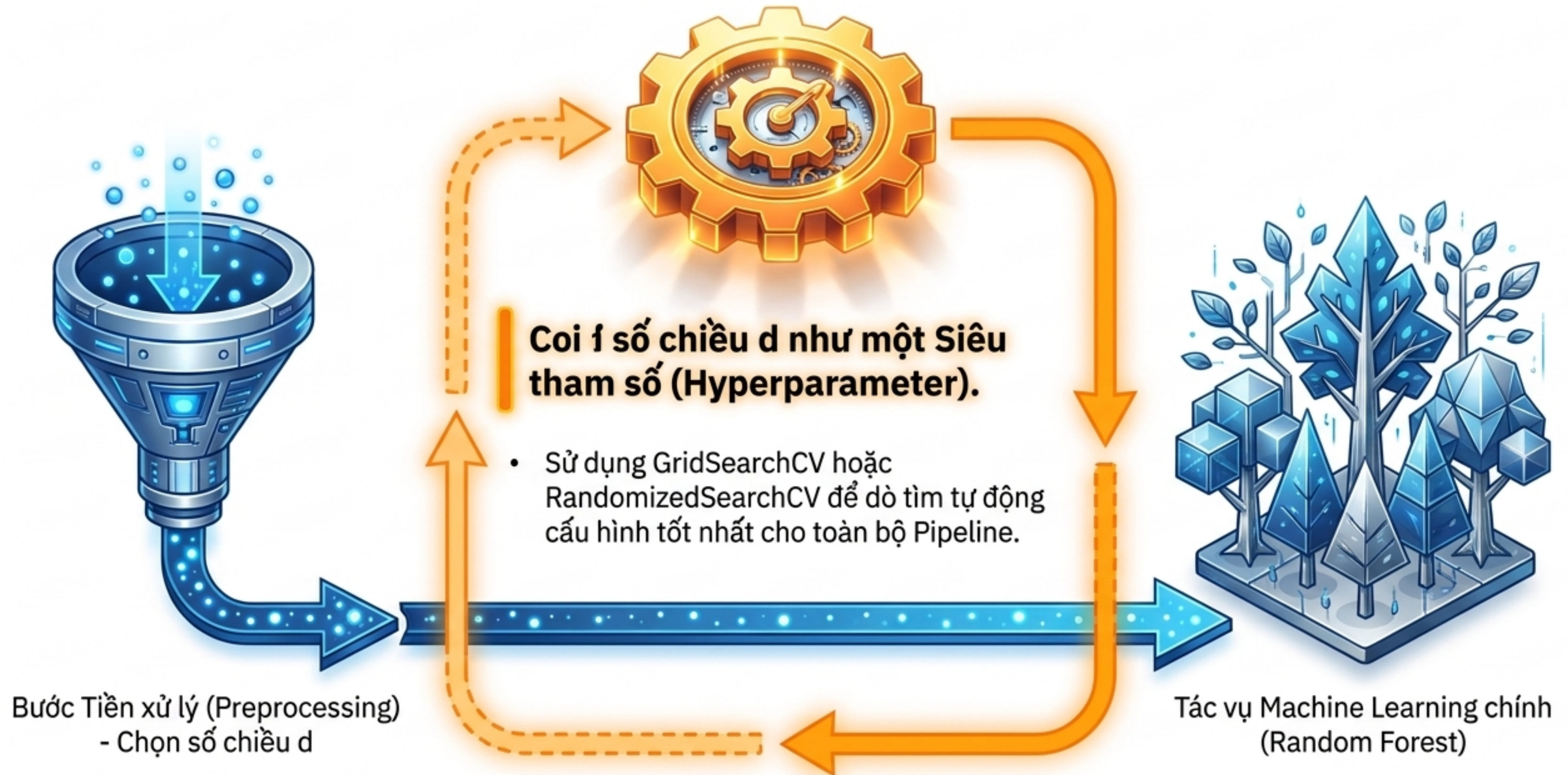
print(pca.n_components_)
# Output: 154
```

Xác định điểm uốn: Vẽ biểu đồ phương sai giải thích



Giúp người phân tích dễ dàng quyết định bằng mắt số chiều cần thiết.

Tối ưu hóa: Điều chỉnh số chiều như một Siêu tham số



Cài đặt RandomizedSearchCV cho Pipeline PCA

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42), RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3, random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])

print(rnd_search.best_params_)
```

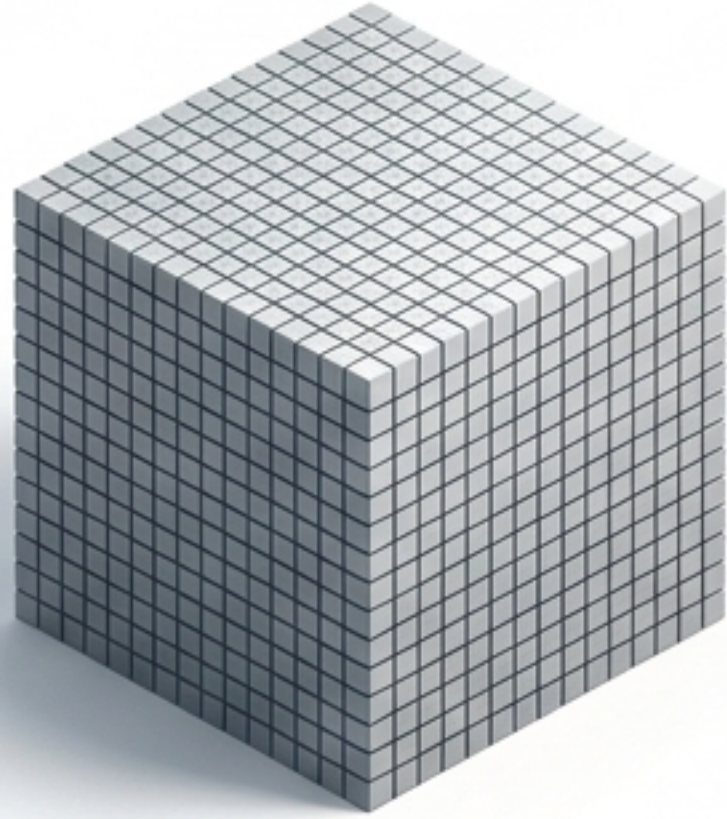


Kết quả bất ngờ!

pca__n_components tối ưu được tìm thấy chỉ là 23 thay vì 154 (khi chạy độc lập).

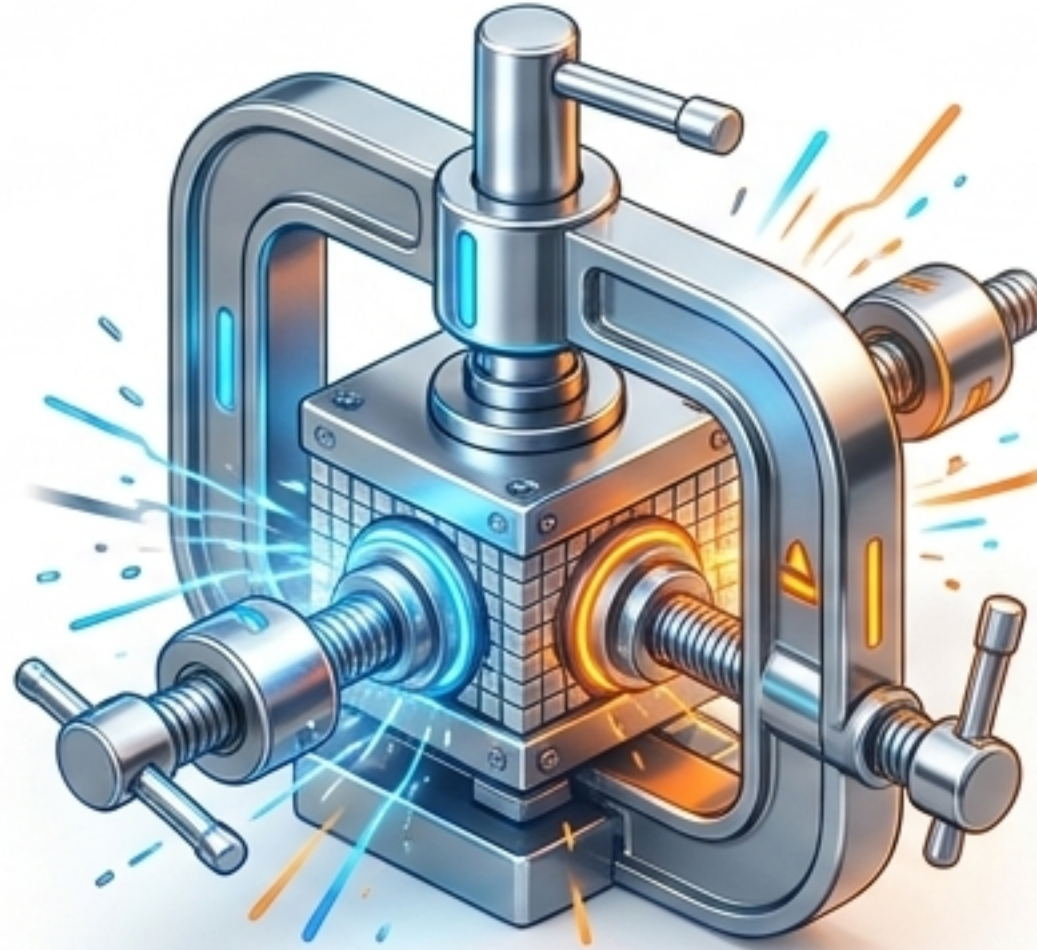
- Mô hình phía sau (Random Forest) hoạt động tốt hơn với dữ liệu được nén chặt hơn.

Bản chất của PCA: Cỗ máy nén dữ liệu



Dữ liệu gốc

784 Đặc trưng (100% kích thước)



Dữ liệu nén (MNIST)

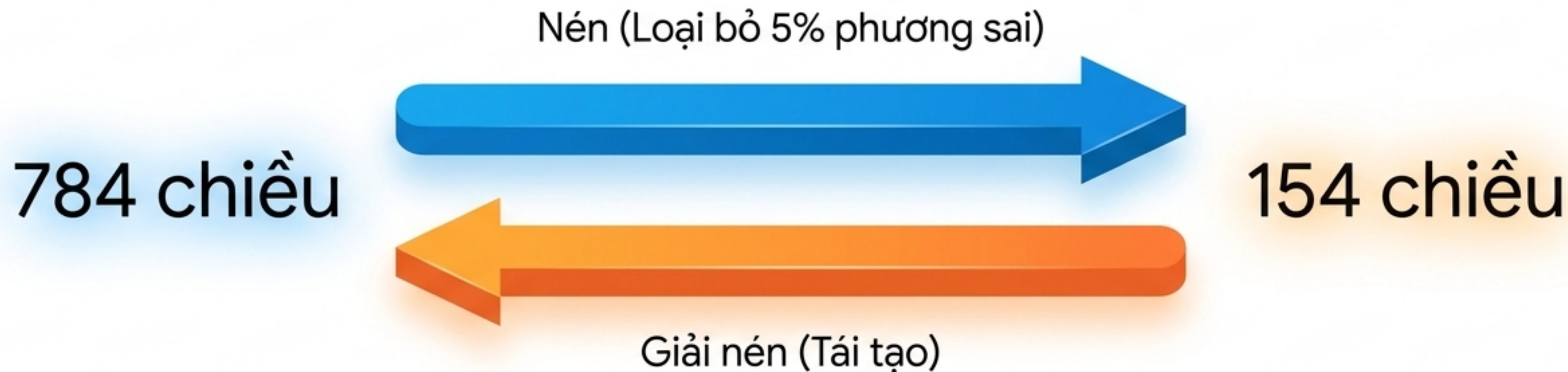
154 Đặc trưng ($\approx 20\%$ kích thước)



Chỉ mất 5% thông tin!

- Tiết kiệm bộ nhớ RAM tuyệt vời.
- Tăng tốc độ thuật toán học máy phía sau vượt trội.

Tái tạo dữ liệu: Nén có hao hụt (Lossy Compression)



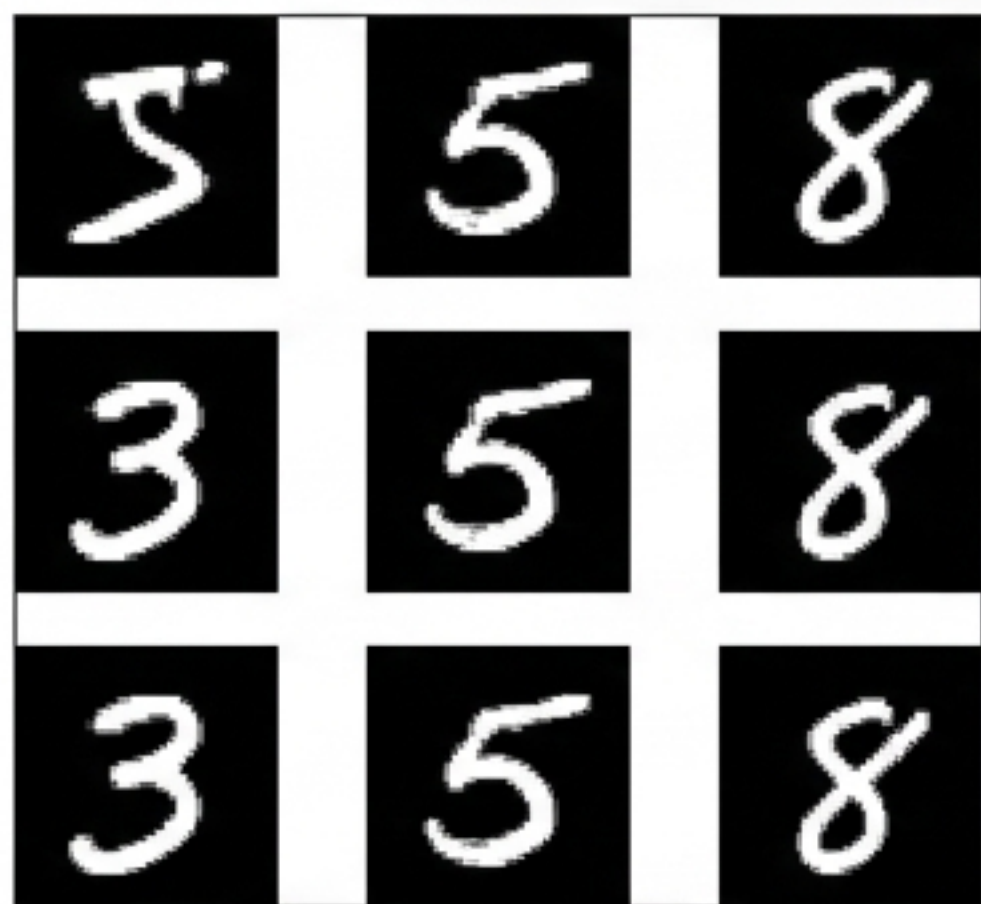
Sai số tái tạo (Reconstruction Error)

Quá trình giải nén sẽ không nhận lại được 100% pixel gốc. Hình ảnh bị mờ nhẹ nhưng phần lớn nội dung vẫn nguyên vẹn.

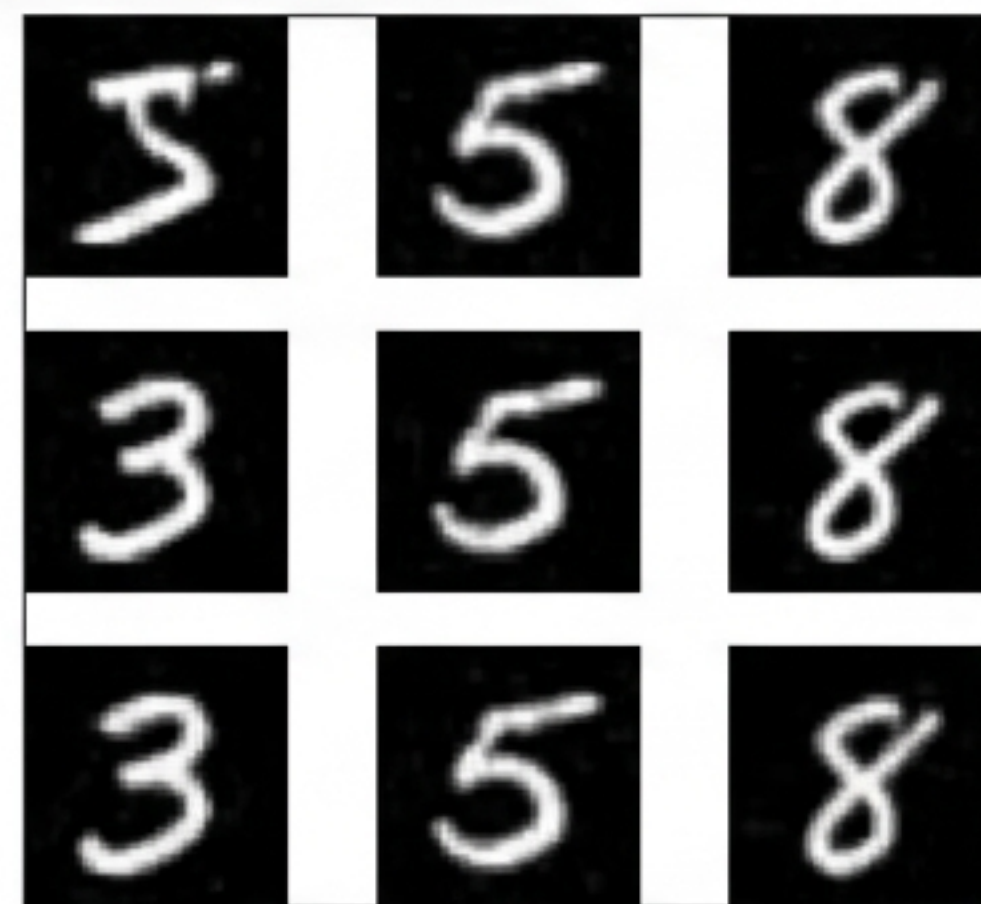
Định nghĩa: Khoảng cách bình phương trung bình giữa Dữ liệu gốc và Dữ liệu giải nén.

Phép biến đổi ngược (inverse_transform)

```
# Sử dụng inverse_transform trên đối tượng PCA  
X_recovered = pca.inverse_transform(X_reduced)
```



Dữ liệu gốc (784 features)



Dữ liệu tái tạo (Từ 154 features)

Mặc dù mờ đi do thiếu 5% phương sai, mắt người (và thuật toán) vẫn dễ dàng nhận diện.

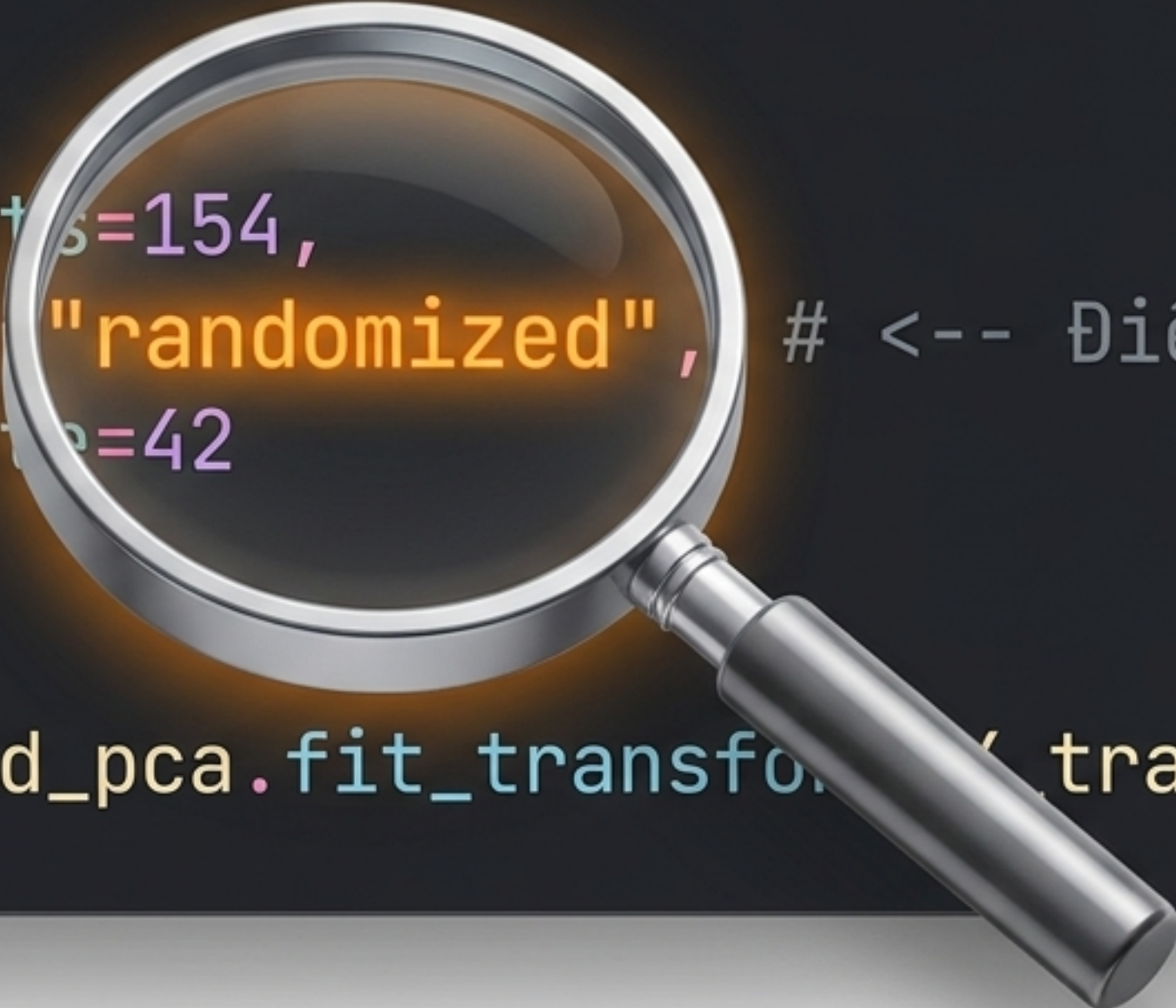
Vượt qua giới hạn tốc độ với Randomized PCA

Thuật toán dùng kỹ thuật ngẫu nhiên hóa để xấp xỉ tìm d thành phần chính đầu tiên một cách nhanh chóng.

SVD Tiêu chuẩn (PCA Đầy đủ)	Randomized PCA
	
$O(m \times n^2) + O(n^3)$	$O(m \times d^2) + O(d^3)$

Nhanh hơn đáng kể khi $d \ll n$ (số chiều giảm xuống nhỏ hơn nhiều so với số chiều ban đầu).

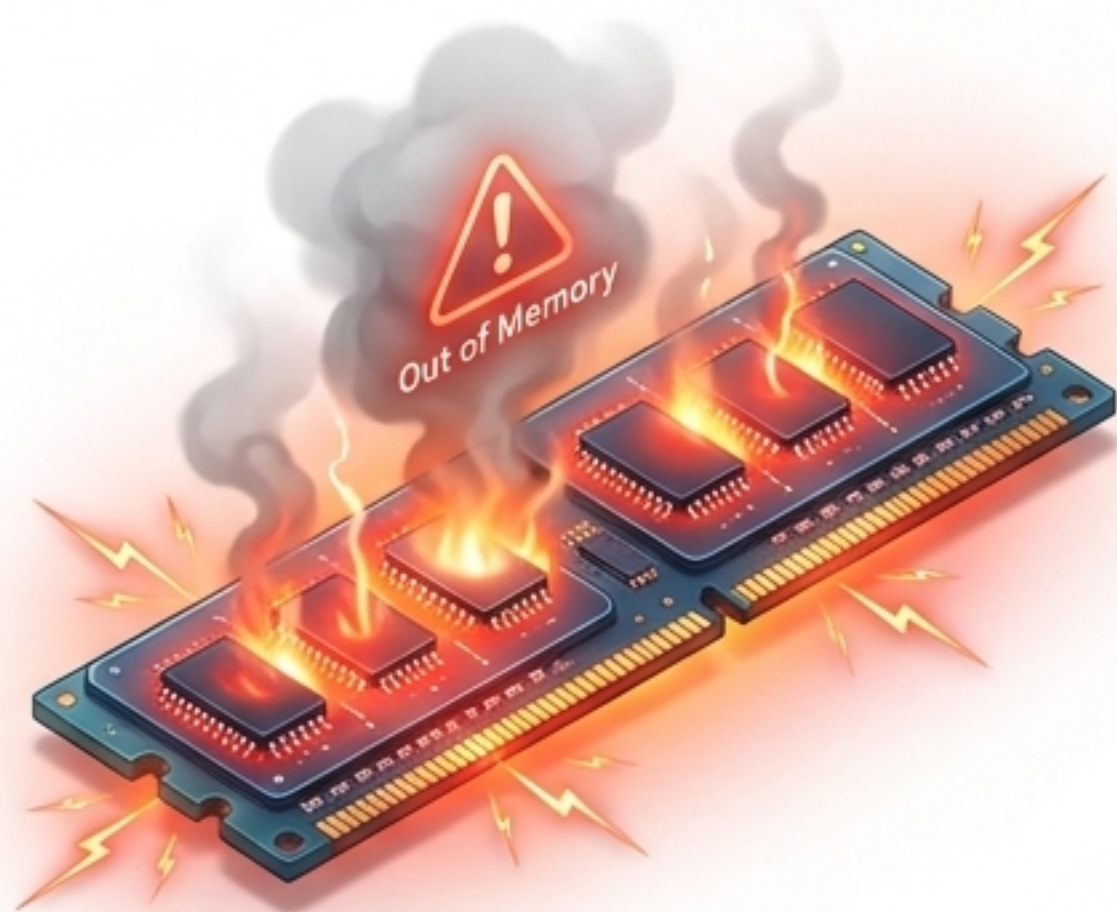
Kích hoạt Randomized PCA trong Scikit-Learn



```
rnd_pca = PCA(  
    n_components=154,  
    svd_solver="randomized", # <-- Điểm mấu chốt  
    random_state=42  
)  
  
X_reduced = rnd_pca.fit_transform(train)
```

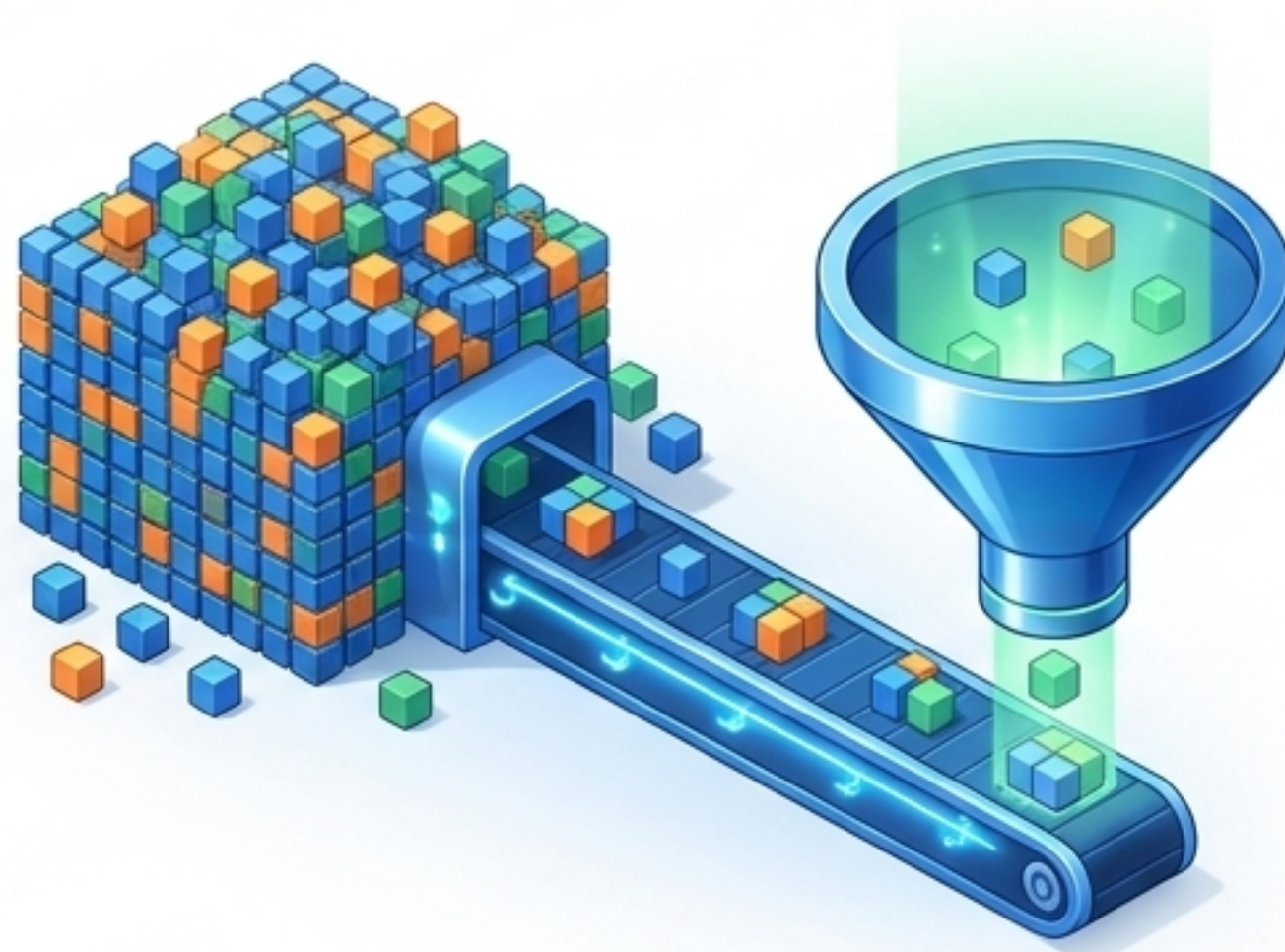
PCA Tăng dần (Incremental PCA): Xử lý dữ liệu khổng lồ

Vấn đề của PCA thông thường



Yêu cầu TOÀN BỘ tập dữ liệu khổng lồ phải nằm gọn trong bộ nhớ (RAM) cùng một lúc.
Rất dễ gây tràn RAM.

Giải pháp: Incremental PCA (IPCA)



Chia nhỏ tập huấn luyện thành các mini-batch
và đẩy vào mô hình từng lô một.

Out-of-core learning

Online Processing

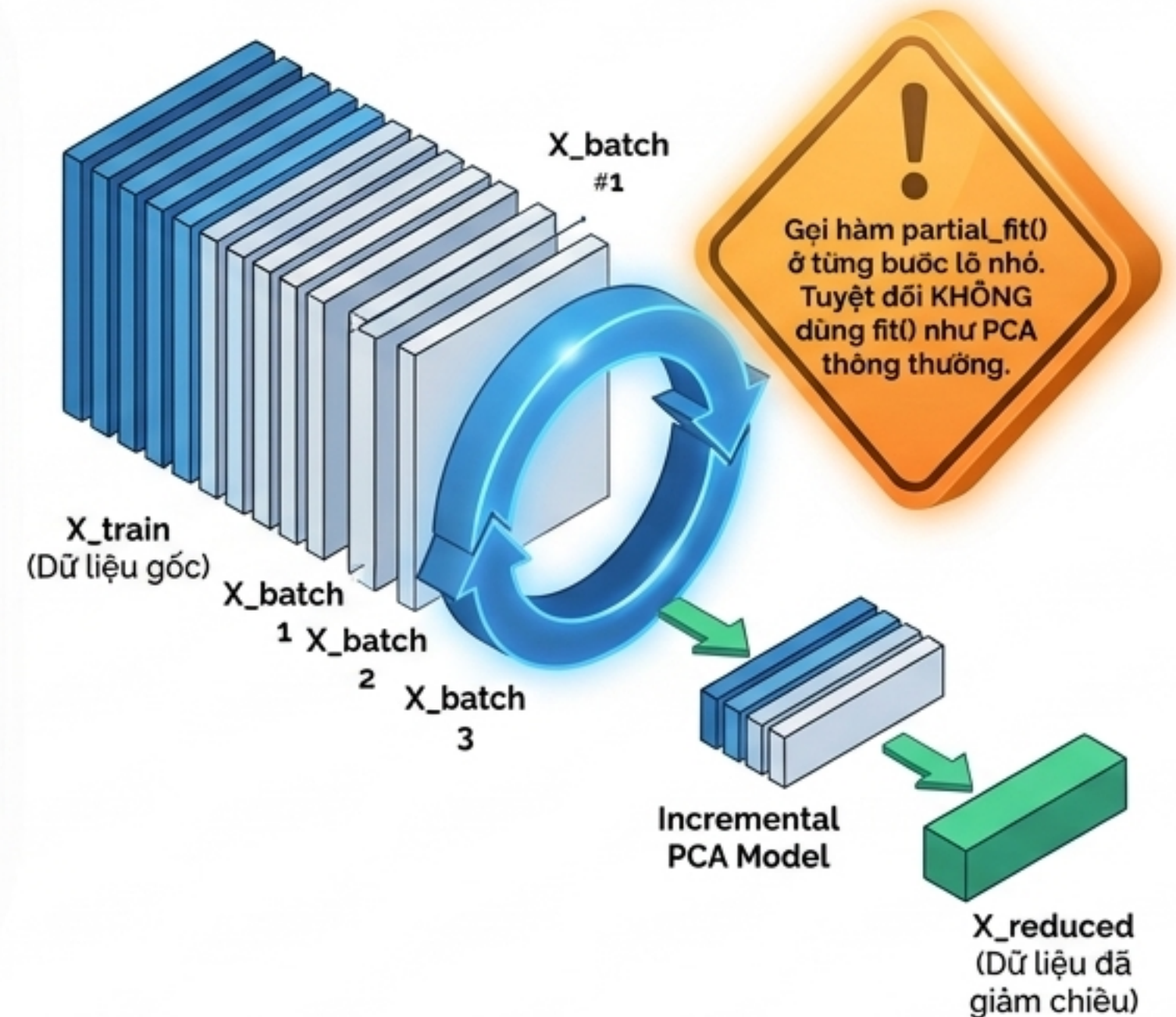
Cài đặt IPCA bằng cách chia mảng (array_split)

```
from sklearn.decomposition import IncrementalPCA
import numpy as np

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch) # <-- Lưu ý

# transform áp dụng cho toàn bộ
X_reduced = inc_pca.transform(X_train)
```



Tối ưu tối đa: Incremental PCA kết hợp memmap

Tập nhị phân lưu trên đĩa (my_mnist.mmap)

Chỉ tải từng khối batch_size vào RAM khi cần



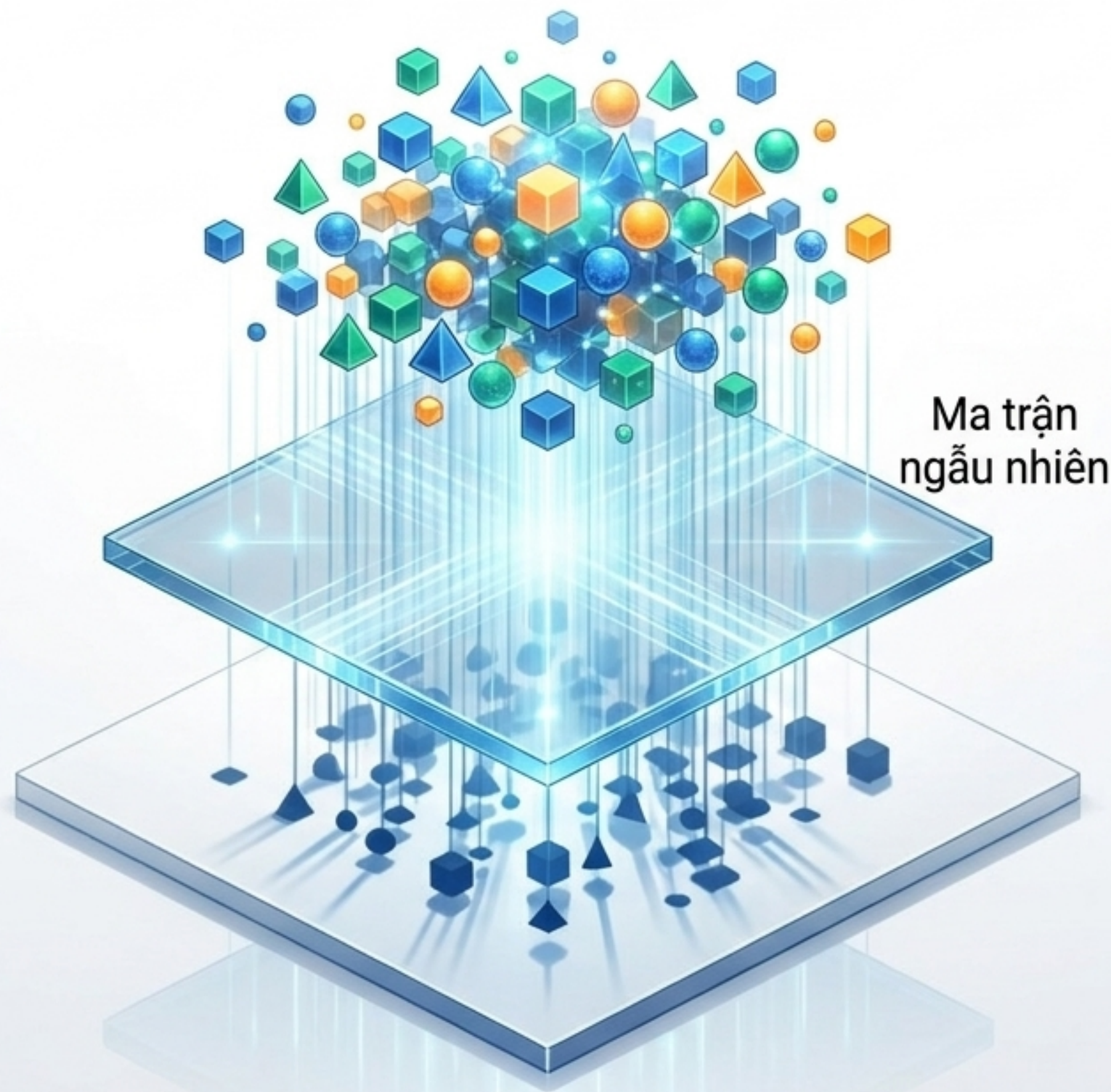
```
# Đọc file đã lưu dưới dạng memmap, mode readonly
X_mmap = np.memmap("my_mnist.mmap", dtype="float32", mode="readonly").reshape(-1, 784)

batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)

# inc_pca tự động hiểu memmap và lấy từng đoạn theo batch_size
inc_pca.fit(X_mmap)
```

✅ Lúc này có thể gọi `fit(X_mmap)` trực tiếp vì IPCA đã tự động hiểu cấu trúc luồng của memmap.

Phương pháp thay thế: Phép chiếu ngẫu nhiên (Random Projection)



Chiếu dữ liệu xuống **không gian thấp** bằng một **ma trận ngẫu nhiên**.

Kỳ diệu thay: Rất có khả năng **bảo toàn khoảng cách** giữa các điểm dữ liệu.

KHÔNG CẦN HUẤN LUYỆN!

Chỉ phụ thuộc vào hình dạng ma trận ban đầu. Rất nhẹ & cực nhanh cho dữ liệu có hàng chục nghìn features.

Cơ sở Toán học: Định lý Johnson-Lindenstrauss (JL)



Có thể chiếu m điểm từ không gian chiều cao xuống không gian d chiều sao cho khoảng cách bình phương giữa mọi cặp điểm chỉ bị méo mó tối đa là một sai số ϵ .

Ví dụ:

$m = 5.000$ điểm

Dung sai $\epsilon = 0.1$

→ Định lý tính được $d \approx 7300$ chiều là đủ an toàn.

KHÔNG quan tâm đến số chiều gốc n .

(Ngay cả khi $n = 1$ triệu). Định lý phá vỡ rào cản tính toán truyền thống.